

TIMETABLES*

Primary Topic: DEP

Course: Data Science, Edition: 2023-1B – Group: 28 – Submission Date: 02-02-2024

BORIS BELCHEV (S2388952) and DAVID GALATI (S2539829)

1 GENERAL QUESTIONS

1.1 Motivation

The efficient structuring of academic timetables is pivotal in shaping the educational experience within universities. This project explores the timetables of the University of Twente from 2013 to 2017, aiming to uncover potential flaws and irregularities that could impact the effectiveness of student and teacher schedules. Research has underscored the significance of ample contact hours in enhancing student engagement and learning outcomes [3], highlighting the crucial role of well-planned timetables in maximizing these interactions. Optimizing timetables is not merely about aligning classes; it also involves the strategic allocation of classrooms and resources across the campus to reduce transit times and ensure the optimal use of time, as studies suggest that high transit times have a negative impact on student satisfaction [4]. By analyzing the schedules of various study programs, this study endeavors to determine whether certain programs inherently benefit from more advantageous timetables, adhering to the Key Performance Indicators (KPIs) set forth by the university. This comparison seeks to identify disparities across programs, delving into how these differences might influence the overall academic quality and student experience. Moreover, the project extends beyond the confines of classroom scheduling to scrutinize the physical layout of the university campus. It questions the efficacy of room allocation and seeks to quantify the extent to which students and teachers navigate between buildings, a factor that significantly influences daily academic logistics. Through a meticulous examination of timetable data over a four-year period, this research aims to reveal patterns and provide insights into optimizing both the structuring of academic schedules and campus logistics. Ultimately, this study is not just an analysis of temporal and spatial dynamics in an academic setting but a crucial step towards refining the educational framework at the University of Twente, thereby enhancing its efficiency and student-centric approach.

1.2 (Business/Research) questions

In our analysis, we focus on the student perspective of academic timetabling, specifically investigating whether current schedules adhere to predefined Key Performance Indicators (KPIs). The aim is to determine compliance with these benchmarks rather than to directly influence or optimize scheduling practices. While certain KPIs are not the subject of this study, we have turned those that are relevant into discrete business questions to assess their alignment with educational objectives. Questions 1–4 are based on Strategy 1's KPIs, whereas questions 5 and 6 are based on Strategy 2. Analysing these collectively will provide an accurate assessment of student workload pressure.

- (1) Which academic programs are meeting the daily 4 to 6 contact hours standard for student engagement in the academic years from 2013 to 2016?
- (2) Are any of our academic programs exceeding the daily limit of 8 hours and 15 minutes of scheduled class time, and if so, which ones?
- (3) How does the frequency of Friday evening classes compare to evening classes scheduled on other weekdays?

* Adapted from the ACM SigConf Template. More information about the template can be found at <https://www.acm.org/publications/proceedings-template>

Authors' address: Boris Belchev (s2388952); David Galati (s2539829).

- (4) Are there any rooms in the UT that on average are below 70% occupation threshold during educational weeks for the academic years 2013, 2014, 2015, and 2016?
- (5) How are the activities distributed between the buildings on campus?
- (6) Which academic programs involve the longest walking distances between classes for teaching activities?

1.3 Source data

1.3.1 Data exploration. Before examining the data, potential quality issues were already considered, particularly the potential absence of crucial timestamps such as dates, start times, and end times, essential for calculating academic activity durations. Concerns were also raised about data consistency, including formatting differences and non-adherence to a standard data format, which could obstruct effective analysis and interpretation. These considerations were pivotal in formulating a strategic approach to data validation and cleaning, aiming to establish a solid basis for the study.

To detect data quality issues, a detailed examination was conducted in Excel. The process began by importing the CSV files into Excel, followed by counting empty cells in each column to identify missing data. Additionally, counting unique entries in specific columns such as *Naam-Activiteit* helped reveal three patterns which could be parsed to extract the program and its type:

- 2013-2014 and 2014-2015 data: *program_code + Bachelor year + activity name*
- 2015-2016 and 2016-2017: *program_code + Module + activity name*
- For master in all files: *program_code + M + activity name*

Upon examination, it was noted that the dataset contains two columns, "Hostkey" and "Hostkey_1." In each row, one of either columns contained values closely resembling course codes, whereas the other contains data of an undetermined nature.

A close review of the date formats was also carried out to ensure their consistency. And it was found that in the same file two different date format were used and also the date included time: Y/m/d H:M:S & m/d/Y H:M:S.

While reviewing the "overview of programs and abbreviations.xlsx" file, a minor issue was identified: the "EE" code is used for both bachelor's and master's Electrical Engineering programs, which might cause confusion in differentiating between the two program levels. Another issue identified was the balance of available data points through the years. We realized that the data points in academic years 2013 and 2014 were containing roughly 45 % more entries which could affect the analysis at a later point. This was important finding for the results later. Another observation made about the data that for some programs we have multiple rows describing the same activity at the same date, time and room but for different course codes. Which in reality should be counted as one activity. Such program is TW (Technische Wiskunde).

1.3.2 Data preparation. Initially, activities data was distributed across four files for various academic years, which were then merged to standardize cleaning and preparation processes. Similar actions were taken for course-teacher files. The "Hostkey" and "Hostkey_1" columns were consolidated into a single "course_code" column by retaining values without the "#" symbol, identified as the course code. Data cleaning for activities dataframe involved removing duplicates across all columns and discarding rows deemed irrelevant for business questions, specifically those lacking start/end times and room details. Furthermore dates in the "Datum" column were standardized to a common format: Y-M-D.

The "Naam-Activiteit" column was split into three: program code, type/year/module, and assumed activity name. The second column was standardized to "Bachelor (B)" or "Master (M)," consolidating variations like B1-B3 and MOD01-MOD12 to "B". Rows lacking a program code were removed for relevance. The first two columns were merged to create a unique program code (e.g., "EE B"), following the structure decided in the program table. Entries that did not match any program listed in the table were either removed or manually verified against the excel files and the UT website to determine their validity as actual programs or mere noise. This involved

assessing the frequency of the program in the activities table and leveraging information from the UT website. Verified programs were then incorporated into the program table.

A duration column was created and the values for each entry were calculated according to start and end time and added. The program data frame was created to represent the program codes and the name of the programs. Because "EE" was not a unique code it was decided that the program code will be recreated by adding the program type "B" and "M". From the courses with *teachers* data frame - teachers were extracted in a separate data frame. Then the courses data frame was joined with the activities dataframe to obtain the program code according to the course code in both tables. The duplicates were removed as well as entries with empty program code.

Building codes were extracted from the initial room codes by taking the first two characters, occasionally omitting leading "Z" characters. A new table with these building codes was created, and geographic coordinates were later added using data from Google Maps. The task of calculating distances between classes required sorting the data by course code, year, date, and start time, and utilizing the geopy library to compute geodesic distances, ensuring accuracy despite data entry duplicates.

1.4 Method

The main methodology steps can be seen in Figure 1. As it can be observed the project started with the assumption that data is not perfect. First it was come up with some general quality issues that data usually has (1.3). Then utilizing Excel data was explored to verify these quality problems and to find new unforeseen ones. Afterwards it was decided on which strategy to focus and what KPIs to be formulated as business questions and answered. Initial cube of the data was designed and it was proceeded with the preparing and cleaning the data using Python. Python was chosen as a technology due to the team familiarity with it. While preparing the data some limits were realized to what initially was decided. More specifically the course had multiple teachers which limited the uniqueness of the course code which on the other hand we wanted to use for connecting the activities with programs through the course code. So a step back was taken and the KPIs rethought. Therefore decision was taken to focus only on the student perspective as initially teachers perspective was included. Data was uploaded to the database after the preparation phase. Then Tableau was used to visualize the data and analysis was undertook and questions were answered using those vizualizations.

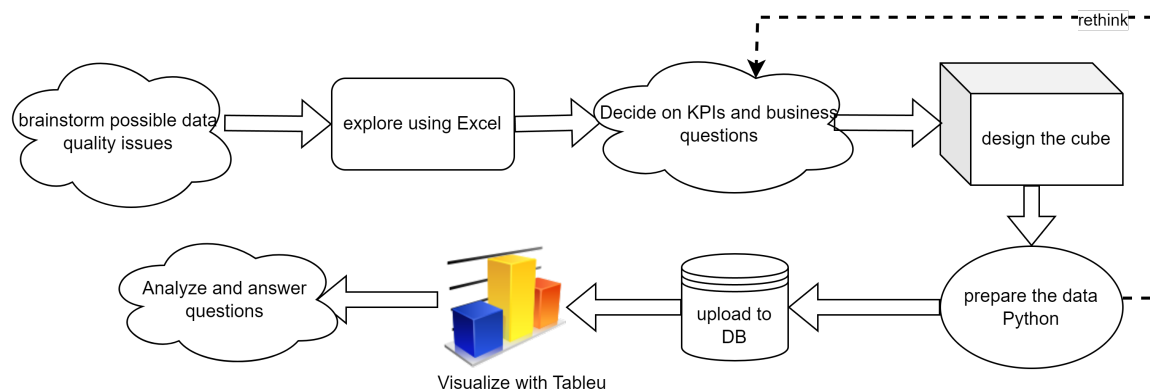


Fig. 1. Methodology steps applied

1.5 Results

1.5.1 Which academic programs are meeting the daily 4 to 6 contact hours standard for student engagement in the academic years from 2013 to 2016? For this visualization new calculated fields were defined in Tableau such as: **DailyContactHours** ({FIXED [Program Name],[Date]: SUM([Duration])}) & **AverageDailyHours** ({FIXED [Program Name], [Year]: AVG([Daily Contact Hours])/3). Due to data imbalance across programs and years, averages were calculated based on actual days available, rather than a standard academic year, to reduce noise from missing or incomplete data. This average was then divided by three to reflect the three bachelor years per program. Analysis of Figure 2 revealed only 6 programs met the minimum contact hours KPI, with Technische Wiskunde (Applied Mathematics) notably exceeding the maximum in 2013, attributed to its role in providing math courses university-wide, affecting program differentiation. In 2014, the same programs increased contact hours beyond the 6-hour limit. Data scarcity in 2015 and 2016 made it difficult to assess adherence to previous standards, highlighting the impact of data imbalance.

Which academic programs are meeting the daily 4 to 6 contact hours standard for student engagement?

Program Name	2013	2014	2015	2016	
Technische Wiskunde	12.92	15.04	8.32	7.54	
Scheikundige Technologie	5.63	7.48	6.96	2.05	
Psychologie	4.21	7.37	3.68	5.07	
Technische Geneeskunde	4.65	4.65	5.22	4.00	
Werktuigbouwkunde	4.40	6.39	4.57	2.45	
Advanced Technology	4.59	6.44	4.48	1.24	
Electrical Engineering	3.56	4.20	2.59	2.63	
International Business Administrat..	2.84	3.56	2.50	2.09	
Creative Technology	2.11	2.83	2.53	3.44	
Industrieel Ontwerpen	3.47	3.47	2.05	1.79	
Technische Informatica	2.50	2.17	2.72	3.07	
Civiele Techniek	1.80	3.56	2.53	1.71	
Biomedische Technologie	2.28	2.94	2.28	1.92	
Technische Natuurkunde	2.64	2.48	1.84	1.32	
Communicatiewetenschap	2.51	2.19	1.70	1.64	
European Public Administration	3.05	1.74	1.77	1.43	
Technische Bedrijfskunde	2.34	2.09	1.53	1.21	
Gezondheidswetenschappen	1.53	1.96	1.65	1.14	
Industrial Engineering	1.51	1.33	0.97	1.30	
Bedrijfsinformatietechnologie	0.87	1.62	0.90	1.42	
Telematics	0.83	0.85	0.93		
Onderwijskunde	1.12	0.70			

green
 < 4 h
 > 6 h
 satisfies

Fig. 2. Average contact hours for Bachelor programs

1.5.2 Are any of our academic programs exceeding the daily limit of 8 hours and 15 minutes of scheduled class time, and if so, which ones? For the visualization here new calculated field were defined again: **Earliest or Latest start-end time** ({FIXED[Program Code],[Date]: MIN/MAX([Start Time])), **MaxCollegeHours** ({FIXED [Program Code],[Date]: MAX((DATEDIFF('minute', [earliest start time], [latest end time]))/60.0)). To identify the day with maximum hours across all years, Figure 3 reveals non-compliance with the KPI across all programs, attributed to extra resits or activities not part of the standard schedule, assumed to be outliers. Despite this, the prevalence of such "long" days suggests a university-wide deviation from the KPI.

Are any of our academic programs exceeding the daily limit of 8 hours and 15 minutes of scheduled class time, and if so, which ones?

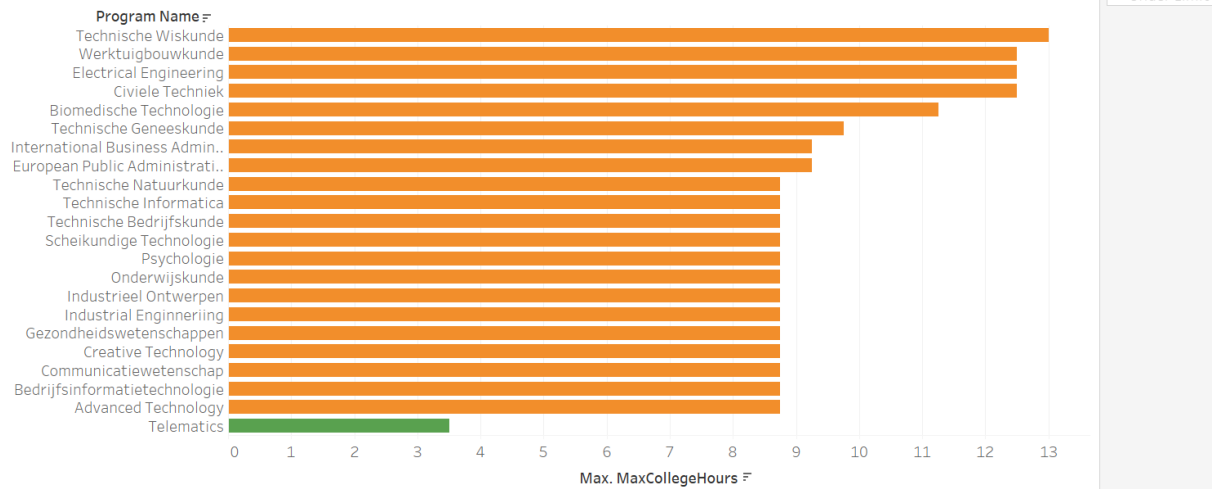


Fig. 3. Programs exceeding the daily limit KPI of 8.25 hours

1.5.3 How does the frequency of Friday evening classes compare to evening classes scheduled on other weekdays?

Figure 4 shows that Friday evening classes account for only 2.6% of all evening classes, the lowest among weekdays. While the KPI may not be fully achievable due to unavoidable scheduling of exams and final presentations on Friday afternoons, the significantly lower frequency of Friday evening classes suggests a deliberate effort to align with the KPI. In conclusion, the data indicates a substantial adherence to the KPI within practical constraints.

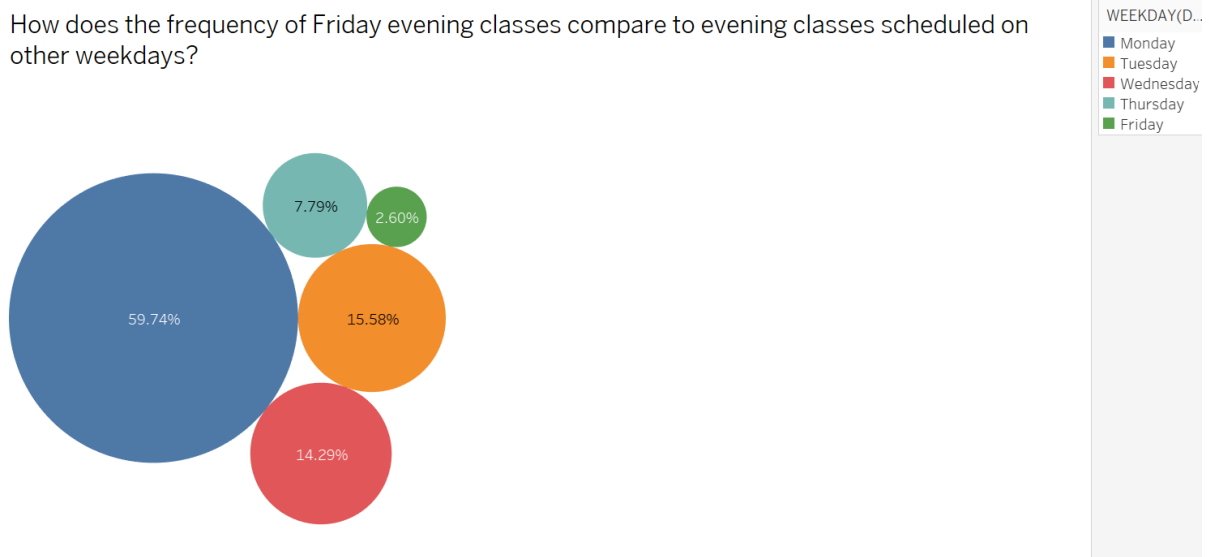


Fig. 4. Distribution of evening classes by weekdays

1.5.4 *Are there any rooms in the UT that on average are below 70% occupation threshold during educational weeks for the academic years 2013, 2014, 2015, and 2016?* To adjust for data imbalances, the **OccupationRate**($\{ \text{FIXED}[\text{room}], [\text{Year}]: \text{SUM}([\text{Duration}]) / (\text{COUNTD}(\text{DATEPART}('week', [\text{Date}])) * 8.75 * 5) \}$) was calculated based on the weeks present in the dataset rather than the total in the actual year. The data indicates that only SC - Sport Centrum in 2013 met the 70% occupation threshold, with Ost-Horst and Carre or Therm rooms nearly reaching it. The majority of rooms fell short of the KPI, suggesting a need for optimization in room utilization. Further investigation revealed that many underutilized rooms are designated for group projects, yet numerous course-related rooms also remain underused (Figure 5).

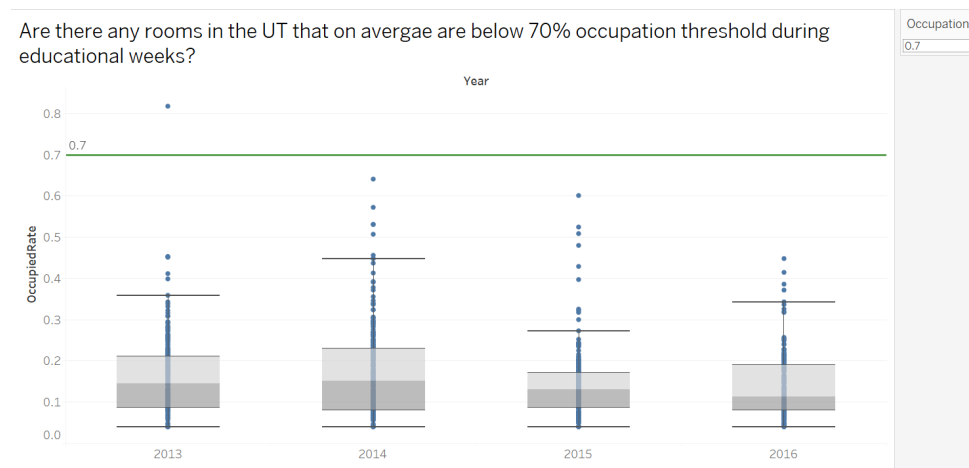


Fig. 5. Box & whiskers plot displaying the annual room occupation rates at UT, benchmarked against a 70% threshold.

1.5.5 *How are the activities distributed between the buildings on campus?* In Figure 6, you can see how the different buildings in the UT campus are used for teaching activities. Knowing the buildings around campus and their capacities. There are clear hot spots of activities in Ravelijn, Carre, and Ost Horst. This is expected since those are the buildings with the highest number of rooms suitable for teaching activities.

1.5.6 *Which academic programs involve the longest walking distances between classes for teaching activities?* In 3 out of the 4 academic years, Psychology had classes that were spread the most between the buildings on campus, with students having to walk a total of 75km across campus over the time span of 2016. This was almost double the amount compared to the second highest, which was Applied Mathematics (Technische Wiskunde). Based on our domain knowledge, this might be caused by the fact that studies with a lower amount of participants could utilize the smaller classrooms in the North West area of the campus, while some of the studies with a higher amount of students are more constrained by the buildings with more high capacity rooms.

Number of activities per building



Fig. 6. Map showing the amount of activities in each of the UT Campus buildings

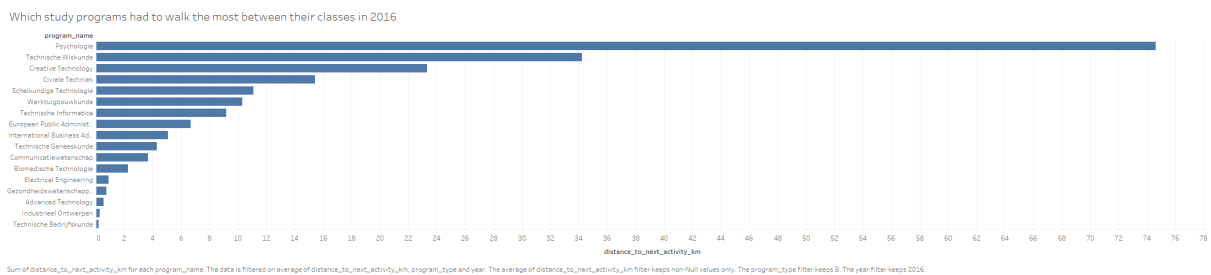


Fig. 7. Which study programs had to walk the most between their classes in 2016

1.6 Reliability of results

1.6.1 Distance data. In our analysis, calculating distances between activities based on their order in the DataFrame and respective coordinates presents a potential reliability concern. This approach assumes activities are sequenced

correctly, reflecting real-world chronological order. Any inaccuracies in the ordering or GPS coordinates directly impact the computed distances. Additionally, straight-line distance calculations may not account for actual walking paths, ignoring obstacles and campus layout. Ensuring data accuracy and considering real-world walking routes are crucial for reliable distance estimations, highlighting the importance of rigorous data validation and the application of domain knowledge to interpret spatial data accurately.

1.7 Technical depth

The DEP course laid the groundwork for our project, especially in initial data handling and analysis. However, challenges arose with geospatial data and distance calculations, pushing us beyond the curriculum. Lacking direct building location data, we leveraged Tableau's geodata capabilities, using Google Maps to obtain longitude and latitude for each building [1, 2]. Initially considering Euclidean distances, we discovered the haversine formula was better suited for calculating geodesic distances on Earth's surface. We utilized the geopy library's geodesic method for this purpose, ensuring accuracy in our spatial analysis. This decision, while seemingly minor, was crucial for the precision of our analysis. We also visualized campus building distribution, enhancing our understanding of University of Twente's spatial dynamics. This exploration into geospatial analysis and visualization extended our skills and emphasized the importance of applying foundational knowledge creatively to meet research needs. Our findings revealed that, despite the short distances, choosing an appropriate method for calculating distances was fundamental to the project's success. Apart from that we leveraged the use of Tableau calculated fields and formulas which was not taught in the course. This helped us mitigate the impact of the imbalanced data.

1.8 Conclusions & recommendations

The primary stakeholder is the scheduling department of the University of Twente, with students being a secondary, yet vital, group of interest. Conclusions indicate that approximately 75-80% of programs, like Biomedische Technologie, need to increase contact hours, unlike Creative Technology which shows improvement over the years. Despite high contact hours in programs such as Technische Wiskunde, the data may not fully reflect actual hours due to factors discussed in Section 1.5. Additionally, many larger classrooms are underused, suggesting a need for the scheduling department to optimize room allocation. What they can improve is that they can monitor and adjust the room occupation rate each month and change it accordingly. To address extended days, often due to extra resits, we recommend scheduling them on Friday evenings. This aligns with current KPI adherence for evening classes and is expected to minimally increase Friday's occupancy by about 1%, as shown in Figure 6.

Activity allocation across campus buildings corresponds to their capacity and room count, with larger buildings like Carre and Horst showing higher usage. This distribution should be considered alongside occupation and travel distance data to streamline student movement. Over four years, Psychology students consistently faced longer walks, notably more than those in Applied Mathematics. Efforts to minimize travel time between consecutive classes are necessary, as distances over a kilometer challenge the feasibility of navigating campus within standard break periods. While not always avoidable, the data suggests a need for more equitable room assignments among programs.

Also a general recommendation about collection future data is to enter the activity with the program code and Module/Bachelor Year in separate columns. Also to have a clear course code column in that table. Apart from that using one format for dates and also whenever the activity is part of the regular curriculum of one program but other programs participate in that to have clear indication of that such as separate rows.

1.9 Reflection

Reflecting on this data science project at the University of Twente, I realize it was quite a stretch from our usual comfort zone. While we weren't new to data science libraries like pandas or numpy, the scale of data processing this project demanded was something we hadn't encountered before. Simple tasks, like merging different dataframes, turned out to be surprisingly time-consuming, pushing us to develop our skills further.

Our intimate knowledge of the university, gained from being students here for over three years, was a real game-changer in this project. Understanding the university's specific jargon helped us accurately map study program codes and building codes. This familiarity with the data was invaluable, particularly when dealing with duplicate entries. We could easily identify which duplicates were likely errors - possibly the same course entries saved for different student groups - and which ones were genuinely distinct activities, like different studies sharing the same courses.

Then there was the challenge of working with geodata, which was entirely new territory for us. Initially, we had no clue about handling location data. But a bit of research led us to use tools like Google Maps to gather GPS coordinates. We divided these into longitude and latitude and added them to our dataframe. Seeing Tableau display these coordinates over a satellite-view map of our familiar campus was both rewarding and enlightening. It wasn't just about learning to handle new types of data; it was about seeing our campus in a whole new light.

In summary, this project wasn't just a technical task; it was a journey full of learning curves. It taught us the importance of domain knowledge in data science, the unexpected complexities of data processing, and the exciting possibilities of geospatial analysis. This experience has not only broadened our understanding of data science but also its practical applications in real-world scenarios. We used ChatGPT to help us with writing complex function and give us advice on formulas in Tableau. We also used it for this report to make paragraphs more succinct and more professionally written.

2 TOPIC SPECIFIC: DEP

2.1 Cube design:

The schema was designed using a normalized approach for the buildings, courses, and program which have separate tables and are referred to with their foreign key in the activities table. For the activities table inlined design was used where the information about the activities are inside the table with the facts. The facts of this cube are duration & distance_to_next_activity_km. Teachers were not included in this cube but if they were it would require a separate bridge table between teacher and courses because multiple teachers relate to multiple courses which creates a bit of redundancy.

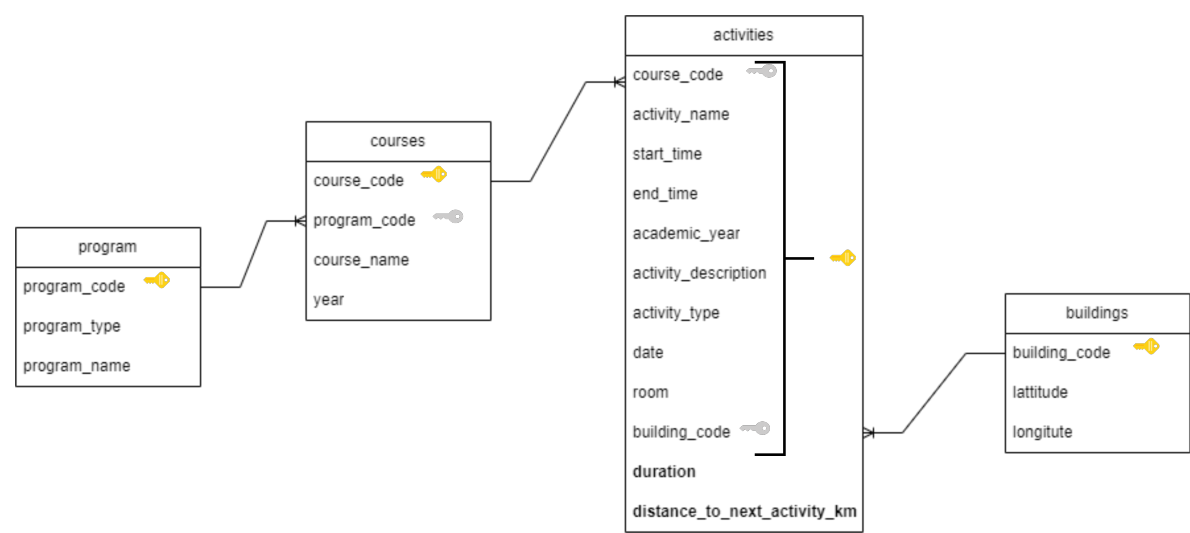


Fig. 8. Final entity-relationship diagram. Facts in bold, PK is yellow key icon, FK is grey key icon

REFERENCES

[1] [n. d.]. A Guide to Mapping and Geographical Analysis in Tableau. <https://www.tableau.com/blog/guide-to-mapping-in-tableau>

[2] [n. d.]. Location Data that Tableau Supports for Building Map Views. https://help.tableau.com/current/pro/desktop/en-us/maps_data.htm

[3] Norma Dawson. 1986. Hours of Contact and Their Relationship to Students' Evaluations of Teaching Effectiveness. *Journal of Nursing Education* 25, 6 (June 1986), 236–239. <https://doi.org/10.3928/0148-4834-19860601-06>

[4] N. Page, Gary Forster-Wilkins, and Mark Bonetzky. 2021. The impact of student timetables and commuting on student satisfaction. *New directions in the teaching of physical sciences* (2021). <https://doi.org/10.29311/NDTPS.V0I16.3793>

A PYTHON CODE

```

1 import pandas as pd # needed for most operation
2 import numpy as np
3 from sqlalchemy import create_engine
4 from geopy.distance import geodesic
5 from datetime import datetime # Import the datetime class from the datetime module
6 import re
7
8 pd.options.mode.chained_assignment = None # default='warn'
9 activity_data_paths = {
10     '2013_2014': 'content/Activiteitenoverzicht_2013-2014_v2.csv',
11     '2014_2015': 'content/Activiteitenoverzicht_2014-2015_v2.csv',
12     '2015_2016': 'content/Activiteitenoverzicht_2015-2016_v2.csv',
13     '2016_2017': 'content/Activiteitenoverzicht_2016-2017_v2.csv'
14 }
15 data_frames = []
16 for year, filename in activity_data_paths.items():
17     df = pd.read_csv(filename)
18     df['academic_year'] = year.split('_')[0]
19     data_frames.append(df)
20     print(f"Loaded data for year: {year}")
21 #merge the activities files
22 merged_data = pd.concat(data_frames, ignore_index=True)
23 print(f"Merged DataFrame shape: {merged_data.shape}")
24
25 # Clean activities without date and room
26 merged_data.dropna(subset=['Datum', 'Zaal-Activiteit'], inplace=True)
27 print("Dropped rows with NaN in 'Datum' and 'Zaal-Activiteit'.")
28 #clean activities without a size of room
29 merged_data = merged_data[merged_data['Grootte'] != 0]
30 print("Dropped rows where 'Grootte' is 0.")
31
32 merged_data.reset_index(drop=True, inplace=True)
33 print(f"After cleaning, the DataFrame shape is: {merged_data.shape}")
34
35 #helper function for combining hostkey column
36 def determine_correct_hostkey(row):
37     if "#" in row['Hostkey']:
38         return row['Hostkey.1']
39     else:
40         return row['Hostkey']
41
42
43 # Apply the function to each row to create a new 'Combined Hostkey' column
44 merged_data['Combined Hostkey'] = merged_data.apply(determine_correct_hostkey, axis=1)
45 merged_data = merged_data.drop(columns=['Hostkey', 'Hostkey.1'])
46 merged_data.dropna(subset=['Combined Hostkey'], inplace=True)
47 merged_data.reset_index(drop=True, inplace=True)
48 print("Merged shape after host keys removed:")

```

```

49 print(merged_data.shape[0])
50 print(merged_data.head())
51 valid_programs_df = pd.read_csv('content/overview of programs and abbreviations.csv')
52 #get the existing source codes
53 valid_program_codes = valid_programs_df['Abbreviation'].unique().tolist()
54
55
56 def parse_activity_name_with_validation(row, valid_codes):
57     """
58     Splits the 'Naam-Activiteit' into three parts based on whitespace and assigns them to
59     'Program Code', 'Year/Module', and 'Activity Name'. Validates the first part against a
60     list of valid program codes.
61     """
62     parts = row['Naam-Activiteit'].split(maxsplit=2)
63     parts += [''] * (3 - len(parts))
64     # Check if the first part is a valid program code
65     program_code = parts[0] if parts[0] in valid_codes else 'GENERIC'
66     # Assign to respective new columns
67     row['Program Code'] = program_code
68     row['Year/Module'] = parts[1] if len(parts) > 1 else ''
69     row['Activity Name'] = parts[2] if len(parts) > 2 else ''
70
71     return row
72
73 parsed_data = merged_data.apply(
74     lambda row: parse_activity_name_with_validation(row, valid_program_codes), axis=1)
75
76 # Drop the original 'Naam-Activiteit' column as it's now redundant
77 parsed_data.drop('Naam-Activiteit', axis=1, inplace=True)
78 reordered_columns = ['Program Code', 'Year/Module', 'Activity Name'] + [col for col in
79     parsed_data.columns if
80     col not in ['Program
81     Code', 'Year/Module',
82     'Activity
83     Name']]
84 parsed_data = parsed_data[reordered_columns]
85 #process and remove rows without program code or type or module
86 parsed_data['Year/Module'].replace('', np.nan, inplace=True)
87 parsed_data = parsed_data.dropna(subset=['Year/Module'])
88 parsed_data = parsed_data.dropna(subset=['Program Code'])
89
90 parsed_data.reset_index(drop=True, inplace=True)
91
92
93 def update_year_module(year_module):
94     if year_module in ['B1-PM', 'MI', 'MOD1', 'MOD2', 'MOD3', 'MOD4', 'MOD01', 'MOD02', 'MOD03',
95         'MOD04', 'M1', 'M2', 'M3', 'M4']:
96         return 'B1'
97     elif year_module in ['MOD5', 'MOD6', 'MOD7', 'MOD8', 'MOD05', 'MOD06', 'MOD07', 'MOD08', 'M5', 'M6', 'M7', 'M8']:

```

```

93         return 'B2'
94     elif year_module in ['B3/PM', 'MOD9', 'MOD10', 'MOD11', 'MOD12', 'MOD09', 'M9', 'M10', 'M11',
95         'M12']:
96         return 'B3'
97     elif year_module in ['M2/M3']:
98         return 'M'
99     else:
100         return year_module
101
102 #transform all MOD1-12 to B or M2/M3 to master and etc
103 parsed_data['Year/Module'] = parsed_data['Year/Module'].apply(update_year_module)
104 parsed_data = parsed_data[parsed_data['Year/Module'].isin(['B1', 'B2', 'B3', 'M'])]
105
106 activities = parsed_data
107
108
109 # Function to convert date strings to 'YYYY-MM-DD' format
110 def convert_to_date_only(value):
111     if isinstance(value, str):
112         try:
113             return datetime.strptime(value, '%Y/%m/%d %H:%M:%S').strftime('%Y-%m-%d')
114         except ValueError:
115             try:
116                 return datetime.strptime(value, '%m/%d/%Y %H:%M:%S').strftime('%Y-%m-%d')
117             except ValueError:
118                 return value
119     return value
120
121 # Function to check if a value is in 'YYYY-MM-DD' format
122 def is_correct_date_format(value):
123     correct_date_format_pattern = re.compile(r'^\d{4}-\d{2}-\d{2}$')
124     return isinstance(value, str) and bool(correct_date_format_pattern.match(value))
125
126 print("Initial DataFrame shape:", activities.shape)
127 # Convert date columns to 'YYYY-MM-DD' format
128 for column in activities.columns:
129     activities[column] = activities[column].apply(convert_to_date_only)
130
131 # Validate the conversion and count correctly formatted dates
132 total_rows = activities.shape[0]
133 correct_format_count = activities['Datum'].apply(is_correct_date_format).sum()
134
135 # Check if all dates are in the correct format
136 all_rows_correct_format = correct_format_count == total_rows
137 print(f"All dates in correct format: {all_rows_correct_format}")
138 print(f"Correctly formatted dates: {correct_format_count} out of {total_rows}")
139
140 # Display the DataFrame shape after processing
141 print("DataFrame shape after processing dates:", activities.shape)

```

```

142
143
144 #updating generic and unknow codes to N/A or add the B or M to produce a program code column
145 def update_program_code(row):
146     # Check if 'Year/Module' is 'GENERIC' or 'UNKNOWN', and return None if so
147     if row['Year/Module'] == 'GENERIC' or row['Year/Module'] == 'UNKNOWN':
148         return None
149     elif row['Year/Module'] == 'PM':
150         return None
151     else:
152         # Otherwise, add the first character of 'Year/Module' to 'Program Code'
153         return row['Program Code'] + ' ' + row['Year/Module'][0]
154
155 # Apply the function to each row and drop rows where the function returns None
156 activities['Program Code'] = activities.apply(update_program_code, axis=1)
157 activities.dropna(subset=['Program Code'], inplace=True)
158 #remove some incorrect programs
159 activities = activities[activities['Program Code'] != 'AM B']
160 activities = activities[activities['Program Code'] != 'PSTS B']
161 activities = activities[activities['Program Code'] != 'GENERIC B']
162 activities = activities[activities['Program Code'] != 'CME B']
163 activities = activities[activities['Program Code'] != 'GENERIC M']
164 activities = activities[activities['Program Code'] != 'NT B']
165 activities = activities[activities['Program Code'] != 'HS B']
166 activities = activities[activities['Program Code'] != 'HMI B']
167 activities = activities[activities['Program Code'] != 'EST B']
168 activities = activities[activities['Program Code'] != 'BA B']
169 activities = activities[activities['Program Code'] != 'TM B']
170 activities = activities[activities['Program Code'] != 'PSY M']
171 activities = activities[activities['Program Code'] != 'TI M']
172 activities = activities[activities['Program Code'] != 'MPS I']
173 activities = activities[activities['Program Code'] != 'ELAN B']
174 activities.reset_index(drop=True, inplace=True)
175
176 #calculate the duration
177 def calculate_duration(row):
178     try:
179         if isinstance(row['Tijd van'], str) and isinstance(row['Tijd tot en met'], str):
180             start_time = datetime.strptime(row['Tijd van'].strip(), '%H:%M:%S')
181             end_time = datetime.strptime(row['Tijd tot en met'].strip(), '%H:%M:%S')
182             return (end_time - start_time).seconds / 3600
183         else:
184             return None
185     except Exception as e:
186         print(f"Error processing row {row.name}: {e}")
187         return None
188
189
190 # Apply the function to the dataframe to create a new 'Duration' column
191 activities['duration'] = activities.apply(calculate_duration, axis=1)

```

```

192 activities.dropna(subset=['duration'], inplace=True)
193 activities.reset_index(drop=True, inplace=True)
194
195 # Display the first few rows to verify the new column
196 print(activities.head())
197 print(activities.shape[0])
198
199 # Dictionary of Dutch to English translations for the column headers
200 translations = {
201     'Program Code': 'program_code',
202     'Year/Module': 'program_type',
203     'Activity Name': 'activity_name',
204     'Beschrijving-Activiteit': 'activity_description',
205     'Activiteitstype': 'activity_type',
206     'Datum': 'date',
207     'Tijd van': 'start_time',
208     'Tijd tot en met': 'end_time',
209     'Grootte': 'size',
210     'Zaal-Activiteit': 'room',
211     'Combined Hostkey': 'course_code'
212 }
213
214 # Apply the translations to the DataFrame's columns
215 activities.rename(columns=translations, inplace=True)
216 activities['course_code'] = activities['course_code'].astype(str)
217 activities.drop_duplicates()
218 activities.reset_index(drop=True, inplace=True)
219 # Display the first few rows to verify the changes
220 print("Program code should change here")
221 print(activities.head())
222 print(activities.shape[0])
223 # Load the datasets
224 teacher_2013_2014_path = 'content/UT_courses_Osiris_with_teacher_2013-2014.csv'
225 teacher_2014_2015_path = 'content/UT_courses_Osiris_with_teacher_2014-2015.csv'
226 teacher_2015_path = 'content/Docentenoverzicht_2015_Osiris.csv'
227 teacher_2016_path = 'content/Docentenoverzicht_2016_Osiris.csv'
228
229 # Read the CSV files
230 teacher_2013_2014 = pd.read_csv(teacher_2013_2014_path)
231 teacher_2014_2015 = pd.read_csv(teacher_2014_2015_path)
232 teacher_2015 = pd.read_csv(teacher_2015_path)
233 teacher_2016 = pd.read_csv(teacher_2016_path)
234
235 # For the 2013-2014 and 2014-2015 datasets, drop the 'Teacher-lastname' column
236 teacher_2013_2014.drop(columns=['Teacher-lastname'], inplace=True)
237 teacher_2014_2015.drop(columns=['Teacher-lastname'], inplace=True)
238
239 # For the 2015 and 2016 datasets, rename 'Medewerker' to 'teacher_id'
240 teacher_2015.rename(columns={'Medewerker': 'teacher_id'}, inplace=True)
241 teacher_2016.rename(columns={'Medewerker': 'teacher_id'}, inplace=True)

```



```

242
243 # For the 2013-2014 and 2014-2015 datasets, rename 'TeacherNr' to 'teacher_id'
244 teacher_2013_2014.rename(columns={'TeacherNr': 'teacher_id'}, inplace=True)
245 teacher_2014_2015.rename(columns={'TeacherNr': 'teacher_id'}, inplace=True)
246
247 # Concatenate all teacher dataframes and drop duplicates based on 'teacher_id'
248 combined_teachers = pd.concat([teacher_2013_2014, teacher_2014_2015, teacher_2015,
249                                teacher_2016])
249 combined_teachers.drop_duplicates(subset='teacher_id', inplace=True)
250 combined_teachers.reset_index(drop=True, inplace=True)
251
252 combined_teachers = combined_teachers[['teacher_id']]
253 combined_teachers = combined_teachers.dropna()
254 print(combined_teachers.head())
255 print(combined_teachers['teacher_id'].nunique())
256 # Renaming columns for the 2013 and 2014 files
257 teacher_2013_2014.rename(columns={
258     'Collegeyear': 'year',
259     'Course': 'course_code',
260     'CourseName': 'course_name',
261     'teacher_id': 'teacher_id'
262 }, inplace=True)
263
264 teacher_2014_2015.rename(columns={
265     'Collegeyear': 'year',
266     'Course': 'course_code',
267     'CourseName': 'course_name',
268     'teacher_id': 'teacher_id'
269 }, inplace=True)
270
271 # Renaming columns for the 2015 and 2016 files
272 teacher_2015.rename(columns={
273     'Collegejaar': 'year',
274     'Cursus': 'course_code',
275     'Cursusnaam': 'course_name',
276     'teacher_id': 'teacher_id'
277 }, inplace=True)
278
279 teacher_2016.rename(columns={
280     'Collegejaar': 'year',
281     'Cursus': 'course_code',
282     'Cursusnaam': 'course_name',
283     'teacher_id': 'teacher_id'
284 }, inplace=True)
285 # Merging all course activities into one DataFrame
286 all_courses = pd.concat([teacher_2013_2014, teacher_2014_2015, teacher_2015, teacher_2016]).
287     drop_duplicates()
288
289 full_merged_file_path = 'content/allCourses.csv'
290 all_courses.to_csv(full_merged_file_path, index=False, encoding='utf-8-sig')

```

```

290 all_courses['program_code'] = '' # Placeholder for program foreign key
291 all_courses['year'] = all_courses['year'].astype(str).str.replace('.0', '', regex=False)
292 all_courses['course_code'] = all_courses['course_code'].astype(str).str.replace('.0', '',
    regex=False)
293 #Get a list of valid course codes from the 'courses' DataFrame
294 valid_course_codes = all_courses['course_code'].unique()
295 print("Valid course codes:")
296 print(valid_course_codes)
297 #Filter the 'activities' DataFrame to only keep rows with valid course codes
298 activities = activities[activities['course_code'].isin(valid_course_codes)]
299 all_courses_with_program = pd.merge(all_courses, activities, on='course_code', how='left')
300 all_courses_with_program = all_courses_with_program.drop(
301     columns=['program_code_x', 'program_type', 'activity_name', 'activity_description', '
    activity_type', 'date'])
302 all_courses_with_program = all_courses_with_program.drop(
303     columns=['start_time', 'end_time', 'size', 'room', 'academic_year', 'duration'])
304 all_courses_with_program.drop_duplicates(subset='course_code', inplace=True)
305 all_courses_with_program.reset_index(drop=True, inplace=True)
306 all_courses_with_program.rename(columns={'program_code_y': 'program_code'}, inplace=True)
307 unique_teacher_count = all_courses_with_program['teacher_id'].nunique()
308 activities.rename(columns={'academic_year': 'year'}, inplace=True)
309 activities = activities[['course_code', 'program_code', 'activity_name', 'start_time', '
    end_time', 'year', 'activity_description',
310     'activity_type', 'duration', 'date', 'room']]
311 all_courses_with_program.dropna(subset=['program_code'], inplace=True)
312 all_courses_with_program.reset_index(drop=True, inplace=True)
313 print(all_courses_with_program.head())
314 print(all_courses_with_program.shape[0])
315 print(activities.head())
316 print(activities.shape[0])
317 file_path = 'content/distinct_buildings_excel.csv'
318 data = pd.read_csv(file_path, delimiter=';')
319
320 # Check if 'location' column exists
321 if 'location' in data.columns:
322     # Split the 'location' column into 'latitude' and 'longitude'
323     data[['latitude', 'longitude']] = data['location'].str.split(',', expand=True)
324
325     data.drop(columns=['location'], inplace=True)
326     data.drop(columns=['Column1'], inplace=True)
327     data.rename(columns={
328         'building': 'building_code'}, inplace=True)
329     # Save the modified DataFrame to a new CSV file
330     new_file_path = 'content/distinct_buildings_modified.csv'
331     data.to_csv(new_file_path, index=False, sep=';')
332
333     print("File processed and saved as:", new_file_path)
334 else:
335     print("The 'location' column was not found in the CSV file.")
336 def extract_building(room_str):

```

```

337     # Check if room_str is None
338     if room_str is None:
339         return None # or return a default value or placeholder if preferred
340     # If the room string starts with 'ZZ ', remove it
341     elif room_str.startswith("ZZ "):
342         room_str = room_str[3:]
343     # Extract the building code (the first two characters)
344     building_code = room_str[:2]
345     return building_code
346 # Apply the function to the 'room' column to create a new 'building' column
347 activities['building_code'] = activities['room'].apply(extract_building)
348 activities = activities.merge(data, left_on='building_code', right_on='building_code', how='
    left')
349 activities.sort_values(by=['course_code', 'program_code', 'year', 'date', 'start_time'],
    inplace=True)
350
351 def calculate_distances(row, next_row):
352     # Check if the next activity meets the criteria to calculate distance
353     if pd.isna(row['latitude']) or pd.isna(row['longitude']) or pd.isna(next_row['latitude'])
    or pd.isna(next_row['longitude']):
354         return None # Return None if any coordinates are NaN
355     if (row['course_code'] == next_row['course_code'] and
356         row['program_code'] == next_row['program_code'] and
357         row['year'] == next_row['year'] and
358         row['date'] == next_row['date']):
359
360         # Check if consecutive activities have the same description
361         if row['activity_description'] == next_row['activity_description']:
362             return None # Set distance to None if descriptions match
363
364         # Calculate and return the distance
365         return geodesic((row['latitude'], row['longitude']),
366                         (next_row['latitude'], next_row['longitude'])).kilometers
367     # Return None if criteria for same course, program, year, and date are not met
368     return None
369
370 # Initialize a list to store calculated distances
371 distances = []
372
373 # Iterate through the DataFrame
374 for i in range(len(activities) - 1):
375     # Calculate distance to next activity if criteria are met
376     distance = calculate_distances(activities.iloc[i], activities.iloc[i + 1])
377     distances.append(distance)
378
379 # Append None for the last activity of each set, as it has no next activity
380 distances.append(None)
381
382 # Assign calculated distances to the DataFrame
383 activities['distance_to_next_activity_km'] = distances

```

```

384 # drop lat and longitude as they are redundant in the db
385 activities = activities.drop('latitude',axis=1)
386 activities = activities.drop('longitude',axis=1)
387 programs = pd.read_csv('content/overview of programs and abbreviations.csv')
388 # Define a function to extract the program type and update the name
389 def extract_program_type(name):
390     program_type = name[0] if name.startswith(('B ', 'M ')) else ''
391     updated_name = name[2:] if program_type else name
392     return program_type, updated_name
393
394 # Apply the function to the 'Name' column
395 programs[['program_type', 'name']] = programs.apply(lambda row: pd.Series(extract_program_type
    (row['Name'])), axis=1)
396 programs = programs.drop(columns=['Name', 'Unnamed: 2'])
397 programs.rename(columns={'Abbreviation': 'program_code'}, inplace=True)
398 programs.rename(columns={'name': 'program_name'}, inplace=True)
399 programs['program_code'] = programs['program_code'] + ' ' + programs['program_type'].str[0]
400 new_row = {
401     'program_code': 'ST M',
402     'program_type': 'M',
403     'program_name': 'Scheikundige Technologie'
404 }
405
406 programs = pd.concat([programs, pd.DataFrame([new_row])], ignore_index=True)
407
408 new_row = {
409     'program_code': 'IEM B',
410     'program_type': 'B',
411     'program_name': 'Industrial Enginneriing'
412 }
413 programs = pd.concat([programs, pd.DataFrame([new_row])], ignore_index=True)
414
415 new_row = {
416     'program_code': 'CS B',
417     'program_type': 'B',
418     'program_name': 'Communication Science'
419 }
420
421 programs = pd.concat([programs, pd.DataFrame([new_row])], ignore_index=True)
422 new_row = {
423     'program_code': 'TEL B',
424     'program_type': 'B',
425     'program_name': 'Telematics'
426 }
427 programs = pd.concat([programs, pd.DataFrame([new_row])], ignore_index=True)
428 new_row = {
429     'program_code': 'TW M',
430     'program_type': 'M',
431     'program_name': 'Technische Wiskunde'
432 }

```

```

433 programs = pd.concat([programs, pd.DataFrame([new_row])], ignore_index=True)
434
435
436 print(programs.head())
437 print(programs.shape[0])
438 driver = 'postgresql'
439 username = 'xxxxxxxxx'
440 dbname = username # it is the same as the username
441 password = 'xxxxxxxxx'
442 server = 'bronto.ewi.utwente.nl'
443 port = '5432'
444 # Creating the connection pool for SQL
445 engine = create_engine(f'{driver}://{username}:{password}@{server}:{port}/{dbname}?
    client_encoding=utf8')
446
447
448 def clean_data(text):
449     if isinstance(text, str):
450         return text.encode('latin1', 'ignore').decode('latin1')
451     else:
452         return text
453
454
455 # Clean the activities in the DataFrame
456 all_courses_with_program['course_name'] = all_courses_with_program['course_name'].apply(
    clean_data)
457 #
458 # # Insert activities from DataFrame into the 'Courses' table in the database
459 combined_teachers.to_sql('teachers', con=engine, schema='timetables', if_exists='append',
    index=False, method='multi')
460 programs.to_sql('programs', con=engine, schema='timetables2', if_exists='append', index=False,
    method='multi')
461 #
462 all_courses_with_program.to_sql('courses', con=engine, schema='timetables', if_exists='append',
    index=False,
463     method='multi')
464 # Close the connection if you're done with it
465 activities = activities.drop_duplicates(subset=['program_code', 'activity_name', '
    activity_type', 'date', 'start_time', 'end_time', 'room', 'year'], keep='first')
466
467 activities = activities.drop('program_code', axis=1)
468
469 # Reset the index after dropping duplicates, if desired
470 activities.reset_index(drop=True, inplace=True)
471 data.to_sql('buildings', con=engine, schema='timetables', if_exists='append', index=False,
    method='multi')
472 activities.to_sql('activities', con=engine, schema='timetables', if_exists='append', index=
    False, method='multi')
473
474 engine.dispose()

```